

IteraDisc – Sistema de Loja Virtual de Discos

1. Introdução

1.1 Descrição Geral do Projeto

O IteraDisc é um sistema de loja virtual desenvolvido para a comercialização de discos de vinil, com foco em simplicidade, organização e separação clara entre front-end e back-end. O projeto foi criado utilizando uma API em C# (.NET) no back-end e uma aplicação em React no front-end, seguindo princípios modernos de desenvolvimento web.

O sistema permite:

- Cadastro e listagem de produtos (discos)
- Gerenciamento de carrinho de compras
- Criação de vendas
- Controle de itens vendidos e descartados
- Comunicação entre front e back por meio de API REST

O IteraDisc foi idealizado tanto como projeto acadêmico quanto como base para um sistema real de e-commerce de nicho.

1.2 Objetivos

Os principais objetivos do projeto são:

- Desenvolver uma API REST estruturada em camadas
 - Implementar um front-end funcional em React
 - Aplicar conceitos de:
 - Arquitetura em camadas
 - Repositórios
 - Serviços
 - Comunicação HTTP
 - Simular o funcionamento real de uma loja virtual
 - Facilitar a manutenção e expansão futura do sistema
-

1.3 Escopo

O projeto contempla:

Incluído:

- Back-end com:
 - Entidades de domínio
 - Repositórios
 - Controllers
 - Endpoints REST
- Front-end com:
 - Listagem de produtos
 - Interface básica de loja
 - Integração com API

Não incluído:

- Sistema de pagamento real
 - Autenticação completa de usuários
 - Controle de estoque avançado
 - Deploy em produção
-

2. Visão Geral do Sistema

2.1 Arquitetura do Sistema

O sistema é dividido em duas grandes partes:

[Front-end (React)] ⇌ [API REST (C# / .NET)] ⇌ [Banco de Dados]

Arquitetura em camadas no Back-end:

- Apresentação (Controllers)
- Serviços
- Repositórios
- Domínio (Entidades)
- Infraestrutura (Banco de Dados)

Essa separação garante:

- Organização do código
- Baixo acoplamento
- Facilidade de manutenção

- Escalabilidade
-

2.2 Funcionalidades do Sistema

Back-end:

- Cadastro de produtos
- Listagem de produtos
- Criação de venda
- Controle de ItemVenda
- Controle de itens descartados
- Endpoints REST documentados via Swagger

Front-end:

- Exibição de produtos na home
 - Consumo da API
 - Interface de loja simples
 - Estrutura modular em componentes React
-

3. Configuração do Ambiente

3.1 Requisitos de Software e Hardware

Back-end:

- Windows 10 ou superior
- .NET SDK 6.0 ou superior
- Visual Studio 2022
- SQL Server (LocalDB ou SQL Server Express)
- Git

Front-end:

- Node.js (versão 18 ou superior)
- npm ou yarn
- Navegador moderno (Chrome, Edge, Firefox)

Hardware mínimo recomendado:

- 8 GB de RAM

- 10 GB de espaço livre em disco
-

3.2 Instruções de Instalação

Back-end:

1. Clonar o repositório do projeto.
2. Abrir a solução no Visual Studio.
3. Restaurar os pacotes NuGet.
4. Configurar a string de conexão no `appsettings.json`.
5. Executar as migrations ou criar o banco manualmente.
6. Executar o projeto.

Acessar o Swagger em:

`https://localhost:xxxx/swagger`

7.

Front-end:

1. Clonar o repositório.
2. Abrir o terminal na pasta do projeto.

Instalar dependências:

`npm install`

3.

Rodar o projeto:

`npm run dev`

4.

Acessar pelo navegador:

`http://localhost:5173`

5.

3.3 Configuração do Ambiente de Desenvolvimento

- O back-end deve estar rodando antes do front-end.
- A URL da API deve estar configurada no front-end (ex: arquivo de serviço Axios).

- Utilizar o Swagger para testar os endpoints manualmente.
-

3.4 Dependências

Back-end:

- ASP.NET Core
- Entity Framework Core
- Swagger (Swashbuckle)

Front-end:

- React
 - Axios
 - Vite
 - React Router
-

4. Desenvolvimento

4.1 Estrutura do Projeto

Back-end:

```
IteraDisc
├── Dominio
│   ├── Entidades
│   └── Repositorio
│       ├── Interfaces
│       └── Implementacoes
├── Servicos
│   ├── Interfaces
│   └── Implementacoes
├── Aplicacao
│   └── Controllers
├── Infraestrutura
└── ContextoBanco
```

Front-end:

```
iteradisc.app
├── src
│   ├── components
│   ├── services
│   ├── pages
│   ├── App.jsx
│   └── main.jsx
```

4.2 Descrição das Camadas

Domínio:

- Contém as entidades principais como Produto, Venda e ItemVenda.

Repositórios:

- Responsáveis pela comunicação direta com o banco de dados.

Serviços:

- Contêm a lógica de negócio do sistema.

Controllers:

- Recebem requisições HTTP e acionam os serviços.

Front-end:

- Camada de apresentação.
 - Consome a API via HTTP.
 - Mostra dados ao usuário.
-

5. API

A API segue o padrão REST e utiliza JSON como formato de comunicação.

Exemplo de Endpoints:

Método	Endpoint	Descrição
--------	----------	-----------

GET	/api/produtos	Lista todos os produtos
GET	/api/produtos/{id}	Busca produto por ID
POST	/api/produtos	Cria novo produto
PUT	/api/produtos/{id}	Atualiza produto
DELETE	/api/produtos/{id}	Remove produto

Método	Endpoint	Descrição
--------	----------	-----------

POST	/api/vendas	Cria uma venda
GET	/api/vendas	Lista vendas

Todos os endpoints podem ser testados no Swagger.

6. Interface do Usuário

O front-end é uma aplicação SPA (Single Page Application) desenvolvida em React.

Funcionalidades principais:

- Listagem de produtos na home.
- Exibição de dados vindos da API.
- Estrutura visual simples simulando uma loja online.
- Separação em componentes reutilizáveis.

Componentes comuns:

- Navbar
- Lista de produtos
- Card de produto
- Serviços de requisição (Axios)

7. Banco de Dados

7.1 Modelo de Entidades

Principais tabelas:

- Produto
 - Produtold
 - Nome
 - Descricao
 - Preco
- Venda
 - Vendald
 - Data
 - Lista
- ItemVenda
 - ItemVendald
 - Produtold
 - Vendido (bool)
 - Descartado (bool)

Relacionamentos:

Venda 1 — * ItemVenda

Produto 1 — * ItemVenda

7.2 Diagrama ERD (representação textual)

PRODUTO (Produtold, Nome, Preco)

|
| 1
|
| *

ITEMVENDA (ItemVendald, Produtold, Vendido, Descartado)

|
| *
|
| 1

VENDA (Vendald, Data)

8. Considerações Finais

8.1 Lições Aprendidas

Durante o desenvolvimento do IteraDisc foi possível aplicar na prática:

- Arquitetura em camadas
 - Separação de responsabilidades
 - Criação de API REST
 - Consumo de API em React
 - Organização de projeto full stack
-

8.2 Melhores Práticas

- Uso de padrões de projeto (Repository e Service)
 - Separação entre domínio e infraestrutura
 - Padronização de endpoints
 - Uso de Swagger para documentação
 - Componentização no React
-

8.3 Próximos Passos

- Implementar autenticação de usuários
 - Criar sistema de pagamento
 - Criar controle de estoque real
 - Melhorar a interface gráfica
 - Criar deploy em nuvem (Azure, Render, Vercel)
-

9. Anexos

9.1 Referências e Recursos Adicionais

- Documentação oficial do .NET:
 - <https://learn.microsoft.com/pt-br/dotnet/>
- Documentação do ASP.NET Core:
 - <https://learn.microsoft.com/pt-br/aspnet/core/>
- Entity Framework Core:
 - <https://learn.microsoft.com/pt-br/ef/core/>

- Swagger / Swashbuckle:
 - <https://github.com/domaindrivendev/Swashbuckle.AspNetCore>
- React:
 - <https://react.dev/>
- Vite:
 - <https://vitejs.dev/>
- Axios:
 - <https://axios-http.com/>
- Padrão Repository:
 - <https://martinfowler.com/eaCatalog/repository.html>
- Arquitetura em Camadas:
 - <https://learn.microsoft.com/pt-br/azure/architecture/guide/architecture-styles/n-tier>

Esses materiais foram utilizados como base conceitual para:

- Estruturação da API
- Organização do código em camadas
- Implementação da comunicação entre front-end e back-end
- Desenvolvimento da interface React

9.2 Links Úteis

Ferramentas de desenvolvimento:

- Visual Studio 2022
<https://visualstudio.microsoft.com/pt-br/>
- SQL Server Express
<https://www.microsoft.com/pt-br/sql-server/sql-server-downloads>
- Node.js
<https://nodejs.org/>
- Git
<https://git-scm.com/>
- Postman (testes de API)
<https://www.postman.com/>
- Swagger UI (já incluso no projeto)
Usado para:
 - Testar endpoints
 - Documentar a API
 - Validar requisições

Ferramentas auxiliares:

- Draw.io (para diagramas ERD):
 - <https://draw.io/>
 - Lucidchart (diagramas):
 - <https://www.lucidchart.com/>
 - Figma (design de interface):
 - <https://www.figma.com/>
-

Recursos de estudo:

- REST API Concepts:
 - <https://restfulapi.net/>
 - HTTP Status Codes:
 - <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Status>
 - JavaScript:
 - <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript>
 - React Router:
 - <https://reactrouter.com/>
-